

WebSockets vs. Server-Sent Events: análisis del modelo de conexión, la escalabilidad horizontal y la reconexión automática

Sofía Sánchez Jiménez

Licenciatura en Informática con énfasis en Sistemas Web
Universidad Nacional de Costa Rica
Campus Benjamín Núñez, Heredia, Costa Rica
sofia.sanchez.jimenez@est.una.ac.cr

Aaron Chaves Baltodano

Licenciatura en Informática con énfasis en Sistemas Web
Universidad Nacional de Costa Rica
Campus Benjamín Núñez, Heredia, Costa Rica
aaron.chaves.baltodano@est.una.ac.cr

Resumen—Este artículo presenta una comparación entre WebSockets y Server-Sent Events (SSE) como alternativas para incorporar comunicación en tiempo real en aplicaciones web. El análisis considera el modelo de conexión, la dirección en que se transmite la información, la escalabilidad horizontal y la recuperación después de una interrupción. También se estudia el uso de balanceadores de carga, sticky sessions y backplanes, como Redis Pub/Sub, para coordinar las conexiones y distribuir eventos cuando existen varias instancias del servidor. La comparación muestra que WebSockets resulta apropiado para funciones que requieren comunicación frecuente en ambas direcciones, como un chat, mientras que SSE ofrece una alternativa más sencilla para notificaciones y actualizaciones enviadas desde el servidor. Además, SSE incorpora reconexión automática mediante EventSource, mientras que en WebSockets este proceso debe ser gestionado por la aplicación. Finalmente, estos criterios se aplican a Chorotega E-Market, donde SSE se propone para informar cambios en el estado de los pedidos y WebSockets para una posible función de chat entre compradores, tiendas y repartidores.

Palabras clave—*websockets, server-sent events, escalabilidad horizontal, reconexión automática, comunicación en tiempo real.*

I. INTRODUCCIÓN

Las aplicaciones web utilizan cada vez más funciones que requieren mostrar información en tiempo real, como chats, notificaciones y cambios en el estado de un pedido. Para implementar estas funciones es necesario seleccionar una tecnología de comunicación adecuada, ya que no todas las aplicaciones necesitan que el cliente y el servidor intercambien información de la misma manera.

WebSockets y Server-Sent Events (SSE) son dos alternativas para mantener una comunicación continua entre el navegador y el servidor. WebSockets permite enviar información en ambas direcciones mediante una misma conexión, por lo que resulta apropiado para funciones como los chats. SSE, en cambio, mantiene una conexión HTTP destinada principalmente al envío de actualizaciones desde el servidor hacia el cliente, lo que puede facilitar funciones como las notificaciones y los cambios de estado [1], [3].

La dirección de la comunicación no es el único aspecto que debe considerarse. Cuando una aplicación funciona sobre varias instancias del servidor, se requiere coordinar las conexiones y distribuir los eventos mediante mecanismos como sticky sessions y backplanes. También es necesario analizar qué sucede cuando la conexión se interrumpe, debido a que SSE ofrece reconexión automática mediante EventSource, mientras que en WebSockets este proceso debe ser gestionado por la aplicación [2], [4], [7].

Este artículo compara WebSockets y SSE según su modelo de conexión, escalabilidad horizontal y proceso de reconexión. A partir de estos criterios, se analiza cuál

alternativa resulta más apropiada para diferentes casos de uso. Finalmente, se aplican los resultados a Chorotega E-Market, considerando SSE para las notificaciones relacionadas con los pedidos y WebSockets para una posible función de chat entre compradores, tiendas y repartidores.

II. MARCO CONCEPTUAL

A. WebSockets

WebSocket es un protocolo que permite mantener una comunicación continua entre un cliente, como un navegador, y un servidor. La conexión comienza mediante una solicitud HTTP en la que el cliente solicita cambiar al protocolo WebSocket. Si el servidor acepta, se establece una conexión que permanece abierta y puede utilizarse para intercambiar información sin crear una nueva solicitud HTTP cada vez que se necesita enviar un mensaje [1], [2].

Una característica importante de WebSocket es que permite comunicación bidireccional. Esto significa que tanto el cliente como el servidor pueden enviar información por la misma conexión mientras esta permanezca abierta. El servidor no necesita esperar una nueva solicitud del cliente para responder, y el cliente también puede enviar datos cuando sea necesario. Esta característica resulta útil en aplicaciones donde existe un intercambio frecuente de información entre ambas partes [1], [2].

Por esta razón, WebSocket es apropiado para funciones como chats, herramientas de colaboración y otras aplicaciones que necesitan interacción continua en tiempo real. Por ejemplo, en un chat el usuario puede enviar un mensaje al servidor y, al mismo tiempo, recibir nuevos mensajes mediante la misma conexión. Esto evita depender de solicitudes HTTP repetidas para comprobar constantemente si existe información nueva [3].

Entre sus principales ventajas se encuentra la posibilidad de mantener un canal bidireccional y reducir la necesidad de realizar solicitudes repetidas durante una comunicación continua. Sin embargo, mantener conexiones abiertas también requiere controlar aspectos como el cierre de la conexión, posibles interrupciones y la coordinación entre servidores cuando la aplicación utiliza varias instancias. Estos aspectos pueden aumentar la complejidad de la solución y deben considerarse según las necesidades de cada aplicación [2], [4].

B. Server-Sent Events

Server-Sent Events (SSE) es una tecnología que permite al servidor enviar información al navegador de manera continua. Para esto, se mantiene abierta una conexión HTTP por la cual pueden enviarse actualizaciones cuando sea necesario, sin que el navegador tenga que solicitarlas repetidamente. En JavaScript, esta conexión se realiza mediante EventSource. SSE es especialmente útil para recibir notificaciones, cambios en el estado de un pedido o información actualizada en tiempo real [5], [6].

La comunicación mediante SSE ocurre principalmente en una dirección: del servidor hacia el navegador. Cuando el usuario necesita enviar información, como confirmar una acción o actualizar un pedido, puede hacerlo mediante solicitudes HTTP o una API REST. SSE también incorpora un mecanismo de reconexión automática cuando la comunicación se interrumpe temporalmente [7]. Además, los eventos pueden incluir un identificador. Al restablecer la conexión, este identificador puede ayudar a indicar cuál fue el último evento recibido y desde dónde deben continuar las actualizaciones [8].

Los mensajes enviados mediante SSE siguen un formato sencillo. Cada evento puede contener la información que recibirá el navegador, un nombre que indique el tipo de evento y un identificador. También puede incluirse un tiempo de espera que indique cuánto debe esperar el cliente antes de intentar reconectarse. Esta estructura permite distinguir diferentes actualizaciones dentro de una misma conexión. Por ejemplo, Chorotega E-Market podría utilizar eventos como `pedido_aceptado`, `pedido_en_preparacion` y `pedido_entregado` para representar los distintos cambios del pedido [8].

C. Escalabilidad horizontal

La escalabilidad horizontal consiste en utilizar varias instancias del servidor para repartir el trabajo. Esto permite atender más usuarios y evita que toda la aplicación dependa de un solo servidor. En SSE, cada conexión permanece asociada con la instancia del servidor que la recibió. Por esta razón, los usuarios pueden quedar conectados a instancias diferentes y una actualización generada en una de ellas no llega automáticamente a las demás [7], [9].

Un balanceador de carga ayuda a repartir las conexiones entre las diferentes instancias. También pueden utilizarse *sticky sessions*, que intentan mantener al usuario conectado con la misma instancia durante su sesión. Esto facilita el manejo de la conexión, pero no permite por sí solo compartir las actualizaciones entre todos los servidores [9].

Para distribuir los eventos se puede utilizar un *backplane*, es decir, un medio compartido que comunica las distintas instancias. Una opción es Redis Pub/Sub: una instancia publica una actualización y las demás la reciben para enviarla a sus respectivos usuarios. Las *sticky sessions* y el *backplane* cumplen funciones diferentes: las primeras mantienen al usuario asociado con una instancia, mientras que el segundo distribuye los eventos entre todas las instancias [7], [9].

Aunque Redis Pub/Sub facilita la distribución de eventos entre las instancias, no conserva automáticamente los mensajes para entregarlos posteriormente. Si una instancia o un usuario se encuentra desconectado cuando se publica una actualización, ese mensaje podría no recibirse. Por esta razón, Redis Pub/Sub puede utilizarse para distribuir las

notificaciones inmediatas, pero la información importante también debe guardarse en una base de datos. De esta manera, después de una desconexión, el sistema puede consultar nuevamente el estado actual del pedido y no depender únicamente de los eventos enviados en tiempo real [10].

El uso de varias instancias también requiere controlar qué ocurre cuando una de ellas deja de estar disponible. Las conexiones mantenidas por esa instancia se cierran y los clientes deben conectarse nuevamente a otro servidor. Las *sticky sessions* pueden conservar temporalmente la relación entre el cliente y una instancia, pero no evitan la desconexión si ese servidor falla. Por esta razón, la aplicación necesita combinar el balanceador de carga, la reconexión del cliente y un mecanismo compartido para distribuir los mensajes entre las instancias [4].

Redis Pub/Sub permite que las instancias compartan mensajes, pero incorpora otro componente que debe configurarse y mantenerse. Además, los mensajes publicados no se conservan para un cliente que se encuentre desconectado en ese momento. Si el chat necesita recuperar mensajes anteriores, estos también deben guardarse en una base de datos. Por lo tanto, Redis Pub/Sub puede utilizarse para distribuir mensajes en tiempo real, pero no debe considerarse un reemplazo del almacenamiento permanente [10].

En el caso de WebSockets, este escenario presenta una dificultad particular, ya que cada conexión activa permanece asociada con la instancia del servidor que la aceptó. Esto puede generar dificultades cuando usuarios conectados a servidores diferentes necesitan intercambiar información, ya que cada instancia conoce directamente las conexiones que mantiene abiertas [4].

Una alternativa para mantener la relación entre un cliente y una instancia son las *sticky sessions*. Este mecanismo permite que las conexiones de un mismo cliente sean dirigidas hacia un servidor determinado. Sin embargo, esto no resuelve por sí solo la comunicación entre diferentes servidores. Si dos usuarios están conectados a instancias distintas, todavía se necesita un mecanismo que permita compartir los mensajes entre ellas [4].

Para resolver este problema puede utilizarse un *backplane*, que funciona como un medio de comunicación compartido entre las distintas instancias. Una opción es Redis Pub/Sub: un servidor puede publicar un mensaje en un canal y las demás instancias suscritas pueden recibirlo y enviarlo a los clientes WebSocket correspondientes [10], [4].

Esta solución permite distribuir la aplicación entre varios servidores, pero también aumenta su complejidad. Además de administrar las conexiones WebSocket, es necesario coordinar las instancias y el sistema utilizado para intercambiar mensajes entre ellas. Por esta razón, la necesidad de escalabilidad horizontal debe considerarse al decidir si WebSockets es adecuado para una aplicación [4].

Por lo tanto, aunque ambas tecnologías pueden utilizarse en arquitecturas con varias instancias, la forma de administrar las conexiones y distribuir los eventos debe considerarse según el modelo de comunicación utilizado.

D. Reconexión automática

Las conexiones en tiempo real pueden interrumpirse por problemas de comunicación, tiempos de espera o fallos temporales del servidor. Por esta razón, es importante que el

sistema pueda restablecer la comunicación sin obligar al usuario a recargar la página. En SSE, el cliente intenta reconectarse automáticamente cuando se pierde temporalmente la conexión [7]. El formato SSE también permite asignar un identificador a cada evento. Cuando el cliente vuelve a conectarse, ese identificador puede ayudar a reconocer cuál fue la última actualización recibida y determinar si es necesario recuperar información pendiente [8].

La reconexión automática facilita la recuperación de la comunicación, pero no garantiza por sí sola que el usuario reciba todos los eventos enviados durante la interrupción. El identificador del último evento permite informar al servidor hasta dónde recibió actualizaciones el cliente, pero el servidor debe conservar la información necesaria para decidir si repite eventos anteriores o envía el estado más reciente. Por lo tanto, la recuperación completa depende tanto de EventSource como de la estrategia utilizada por el backend para almacenar y consultar la información [7], [8].

En WebSockets, cuando una conexión se pierde o se cierra, el cliente recibe el evento correspondiente al cierre de la comunicación. Sin embargo, la API básica de WebSocket no restablece por sí sola una nueva conexión, por lo que la aplicación debe detectar la desconexión e iniciar nuevamente el proceso de conexión cuando sea necesario [2].

Ante cierres anormales, los nuevos intentos de conexión pueden realizarse utilizando tiempos de espera para evitar que numerosos clientes intenten reconectarse continuamente al mismo tiempo. Para intentos posteriores, el protocolo contempla aumentar progresivamente este tiempo de espera mediante estrategias como exponential backoff [1].

Restablecer la conexión no garantiza por sí mismo recuperar los mensajes enviados durante la interrupción. Si la aplicación necesita conservar esa información, debe incorporar un mecanismo adicional que permita identificar qué mensajes fueron recibidos y recuperar posteriormente los que hayan quedado pendientes.

La aplicación también debe decidir cuándo intentar nuevamente la conexión y cuántos intentos realizar. Si una gran cantidad de clientes intenta reconectarse al mismo tiempo después de una interrupción del servidor, se puede generar una carga adicional. Para reducir este problema, pueden utilizarse tiempos de espera que aumenten progresivamente entre cada intento. De esta manera, los clientes no envían solicitudes de reconexión de forma continua mientras el servicio todavía se está recuperando [1].

Volver a establecer la conexión WebSocket no significa que los mensajes enviados durante la interrupción se recuperen automáticamente. Si esta información es importante, el sistema debe asignar identificadores a los mensajes o guardar su estado para reconocer cuáles fueron recibidos. Después de la reconexión, el cliente podría solicitar nuevamente los mensajes pendientes o consultar el estado actual mediante una API. Esta lógica debe ser diseñada por la aplicación, debido a que la interfaz básica de WebSocket no proporciona por sí sola un mecanismo para recuperar la información perdida [2].

Por lo tanto, WebSockets permite recuperar la comunicación después de una interrupción, pero esta lógica debe ser gestionada por la propia aplicación. Esto implica un mayor control sobre la forma en que se realiza la reconexión

y la recuperación de información, aunque también agrega trabajo adicional en comparación con mecanismos que incorporan la reconexión desde su propia interfaz.

III. ANÁLISIS COMPARATIVO

TABLA I. COMPARACIÓN ENTRE WEBSOCKETS Y SERVER-SENT EVENTS

Criterio	WebSockets	SSE
Dirección	Bidireccional: cliente y servidor pueden enviar información.	Unidireccional: del servidor hacia el cliente.
Conexión	Conexión persistente entre cliente y servidor.	Conexión HTTP que permanece abierta
Reconexión	Debe ser gestionada por la aplicación después de una desconexión.	Automática mediante EventSource
Recuperación	Requiere mecanismos adicionales para recuperar mensajes perdidos durante una desconexión.	Puede utilizar identificadores de eventos
Escalabilidad	Requiere coordinar las conexiones cuando existen varias instancias del servidor.	Varias instancias y un backplane para distribuir los eventos
Sticky sessions	Pueden mantener al cliente asociado con una instancia, pero no distribuyen mensajes entre servidores.	Pueden mantener al cliente asociado con una instancia
Backplane	Puede utilizar Redis Pub/Sub para distribuir mensajes entre diferentes instancias.	Puede utilizar Redis Pub/Sub para distribuir eventos entre diferentes instancias.
Complejidad	Mayor cuando se requiere reconexión, recuperación de mensajes y varias instancias.	Menor cuando solamente se reciben actualizaciones
Caso recomendado	Chat y aplicaciones con intercambio continuo de información en	Notificaciones y cambios de estado

Criterio	WebSockets	SSE
	ambas direcciones.	
Limitación principal	Puede añadir complejidad innecesaria cuando solo se necesita comunicación del servidor hacia el cliente.	No permite comunicación bidireccional por el mismo canal

SSE presenta una ventaja cuando la aplicación necesita enviar información principalmente desde el servidor hacia los usuarios. Al utilizar una conexión HTTP y ofrecer reconexión automática mediante EventSource, reduce parte de la lógica que debe programarse en el cliente. Esto lo convierte en una alternativa apropiada para notificaciones, actualizaciones de estado y otros casos en los que el usuario no necesita responder por el mismo canal [5], [6].

Su principal limitación es que la comunicación ocurre en una sola dirección. Cuando el usuario necesita enviar información al servidor, debe utilizar otro mecanismo, como una API REST [6], [5]. Por esta razón, SSE por sí solo no sería apropiado para un chat bidireccional, aunque sí puede utilizarse para informar cambios en el estado de un pedido.

Cuando se utilizan varias instancias del servidor, es necesario coordinar la distribución de los eventos. Las sticky sessions pueden mantener cada conexión asociada con una instancia, mientras que un backplane, como Redis Pub/Sub, permite compartir las actualizaciones entre las diferentes instancias [7], [9].

En cuanto al mantenimiento, SSE puede reducir el código necesario en el navegador porque EventSource incorpora la reconexión automática. Sin embargo, esta facilidad no elimina la necesidad de controlar los eventos en el servidor. La aplicación debe definir cuáles actualizaciones se almacenan, durante cuánto tiempo pueden recuperarse y qué debe ocurrir si el cliente vuelve a conectarse después de un periodo prolongado. También debe considerarse que un backplane distribuye eventos entre las instancias, pero no reemplaza el almacenamiento permanente de la información [7], [8], [10].

WebSockets resulta más apropiado cuando la aplicación necesita una comunicación constante en ambas direcciones. Al permitir que cliente y servidor envíen información por el mismo canal, se adapta a funciones como chats, colaboración en tiempo real y otras interacciones en las que ambas partes necesitan intercambiar datos de manera continua [1], [3].

Esta capacidad también implica una mayor responsabilidad para la aplicación. Si la conexión se interrumpe, la reconexión y la posible recuperación de mensajes deben gestionarse mediante mecanismos adicionales. Además, cuando existen varias instancias del servidor, cada conexión WebSocket permanece asociada con una instancia, por lo que puede ser necesario utilizar sticky sessions para mantener esa relación y un backplane, como Redis Pub/Sub, para distribuir mensajes entre servidores [2], [4], [10].

En cuanto al mantenimiento, WebSockets requiere que el equipo controle el ciclo completo de la conexión. Esto incluye detectar cierres, programar nuevos intentos, evitar reconexiones continuas, recuperar mensajes pendientes y eliminar conexiones que ya no se utilizan. Estas tareas ofrecen mayor control sobre la comunicación, pero también aumentan la cantidad de lógica que debe mantenerse y probarse. Esta complejidad se justifica cuando existe un intercambio frecuente en ambas direcciones, pero puede ser innecesaria cuando el servidor solamente necesita enviar notificaciones [1], [2].

Por estas razones, WebSockets ofrece mayor flexibilidad cuando existe un intercambio bidireccional frecuente, pero esa capacidad puede introducir complejidad innecesaria cuando la aplicación únicamente necesita enviar actualizaciones desde el servidor hacia el usuario. La elección entre ambas tecnologías debe considerar la dirección de la comunicación, la necesidad de recuperar la conexión y la forma en que la aplicación será distribuida entre varias instancias.

IV. APLICACIÓN A CHOROTEGA E-MARKET

En Chorotega E-Market, SSE puede utilizarse para informar a los usuarios sobre los cambios en el estado de sus pedidos. Por ejemplo, el comprador podría recibir avisos cuando la tienda acepta el pedido, inicia su preparación, lo entrega al repartidor o completa la entrega. Esta información se origina en el servidor y el usuario solamente necesita recibirla, por lo que corresponde a un caso de comunicación unidireccional [5], [6].

El navegador mantendría una conexión SSE con el backend de NestJS mediante EventSource. Cuando cambie el estado de un pedido, el backend actualizaría la información en PostgreSQL y enviaría una notificación a los usuarios relacionados. Las acciones de la tienda, el comprador o el repartidor continuarían enviándose mediante la API REST, mientras que SSE se utilizaría para recibir las actualizaciones [7].

Si el backend se ejecuta en varias instancias, Redis Pub/Sub podría utilizarse como backplane para compartir los eventos entre ellas. Cada instancia recibiría las actualizaciones y las enviaría a los usuarios que mantiene conectados. Un balanceador de carga repartiría las conexiones y podría utilizar sticky sessions para conservar la relación entre el usuario y una instancia determinada [7], [9].

Si la conexión se interrumpe, EventSource intentaría recuperarla automáticamente. Después de la reconexión, el backend podría consultar PostgreSQL para obtener el estado actual del pedido y utilizar la bitácora de MongoDB para revisar los cambios anteriores. Además, si se asignan identificadores a los eventos, el cliente podría indicar cuál fue la última actualización recibida. De esta manera, el usuario podría volver a sincronizarse aunque no haya recibido alguna actualización durante la interrupción [6], [7], [8].

Para Chorotega E-Market se propone utilizar SSE únicamente para las notificaciones relacionadas con el estado de los pedidos. PostgreSQL conservaría el estado actual, mientras que MongoDB mantendría la bitácora de los cambios realizados. Redis Pub/Sub se utilizaría para distribuir las notificaciones entre las instancias, pero no sería la fuente principal de la información. Esta separación permitiría que el usuario consulte nuevamente el estado del pedido aunque una notificación no se haya recibido durante una desconexión.

Esta propuesta requiere implementar un punto de conexión SSE en NestJS, identificar los usuarios relacionados con cada pedido y controlar la eliminación de las conexiones cerradas. También sería necesario configurar Redis si el backend utiliza varias instancias. A cambio, el frontend tendría una forma sencilla de recibir notificaciones y no necesitaría consultar repetidamente el servidor para comprobar si el pedido cambió de estado.

En Chorotega E-Market, WebSockets podría utilizarse si se incorpora una función de chat entre compradores, tiendas y repartidores. En este caso, las personas involucradas necesitan enviar y recibir información de forma continua, por ejemplo, para realizar una consulta sobre un producto, coordinar una entrega o aclarar una indicación relacionada con la ubicación. La comunicación bidireccional permitiría realizar este intercambio mediante una misma conexión, sin depender de solicitudes repetidas para comprobar si existen mensajes nuevos [1], [3].

Si esta funcionalidad tuviera que funcionar sobre varias instancias del servidor, también sería necesario considerar la coordinación de las conexiones. Los usuarios podrían estar conectados a servidores diferentes, por lo que un mecanismo compartido, como Redis Pub/Sub, podría utilizarse para distribuir los mensajes hacia la instancia donde se encuentra conectado cada participante [4], [10].

Sin embargo, WebSockets no sería necesario para todas las funciones en tiempo real de Chorotega E-Market. Su uso se justificaría principalmente cuando ambas partes necesiten enviar y recibir información de manera frecuente. Incorporarlo en funciones donde solamente se requiere informar una actualización desde el servidor hacia el usuario añadiría mecanismos de conexión, reconexión y coordinación que podrían no ser necesarios. Por esta razón, la selección de la tecnología debe responder al tipo de interacción que requiere cada funcionalidad.

Por lo tanto, en Chorotega E-Market no se propone utilizar WebSockets para todas las actualizaciones en tiempo real. Su implementación se reservaría para una posible función de chat, donde compradores, tiendas y repartidores necesiten enviar y recibir mensajes por el mismo canal. Esta decisión evita incorporar la lógica de conexión, reconexión y distribución de mensajes en funciones donde solamente se necesita informar un cambio desde el servidor.

Implementar el chat requeriría crear un punto de conexión WebSocket en NestJS, identificar a los usuarios participantes, guardar los mensajes importantes y utilizar Redis Pub/Sub si existen varias instancias del backend. Esto aumentaría el trabajo de implementación y mantenimiento, pero permitiría una comunicación bidireccional más directa entre las personas relacionadas con un pedido [4], [10].

V. CONCLUSIONES

WebSockets y Server-Sent Events permiten incorporar comunicación en tiempo real, pero responden a necesidades diferentes. WebSockets ofrece comunicación bidireccional, por lo que resulta apropiado cuando el cliente y el servidor necesitan enviar información continuamente. SSE se concentra en el envío de actualizaciones desde el servidor hacia el cliente y puede ser una alternativa más sencilla cuando el usuario solamente necesita recibir información.

La reconexión representa otra diferencia importante. EventSource intenta recuperar automáticamente una conexión

SSE después de una interrupción y los identificadores de eventos pueden ayudar a determinar cuál fue la última actualización recibida. En WebSockets, la aplicación debe implementar la lógica necesaria para restablecer la conexión y recuperar los mensajes que no se recibieron durante la desconexión.

La escalabilidad horizontal requiere planificación en ambas tecnologías. Las sticky sessions pueden mantener al usuario asociado con una instancia determinada, pero no distribuyen información entre servidores. Para compartir los eventos se necesita un mecanismo adicional, como un backplane basado en Redis Pub/Sub. Por lo tanto, ambos mecanismos cumplen funciones diferentes y pueden utilizarse de manera complementaria.

En Chorotega E-Market, SSE sería una opción adecuada para informar los cambios en el estado de los pedidos, mientras que WebSockets tendría mayor utilidad en una posible función de chat entre compradores, tiendas y repartidores. En consecuencia, ninguna tecnología es superior en todos los casos. La elección debe basarse en la dirección de la comunicación, las necesidades de reconexión, la recuperación de información y la forma en que la aplicación será distribuida entre varias instancias.

Como límite del análisis, no se realizaron pruebas de rendimiento con una cantidad elevada de conexiones simultáneas ni se comparó el consumo de recursos de ambas tecnologías en un entorno de producción. Por esta razón, las conclusiones se basan en el modelo de comunicación, la documentación técnica y los estudios consultados.

Como trabajo futuro, se podría implementar un prototipo en Chorotega E-Market para medir el comportamiento de SSE y WebSockets con varias instancias del backend. También sería conveniente evaluar la recuperación de eventos, el uso de Redis Pub/Sub y la respuesta del sistema cuando una instancia deja de estar disponible.

REFERENCIAS

- [1] I. Fette and A. Melnikov, "The WebSocket Protocol," RFC 6455, Internet Engineering Task Force, Dec. 2011, doi: 10.17487/RFC6455.
- [2] WHATWG, "WebSockets Living Standard." [Online]. Available: <https://websockets.spec.whatwg.org/>. Accessed: Oct. 6, 2026.
- [3] K. J. Gracia Orejuela, M. B. Gorozabel Bazurto, and W. G. Chango Sailema, "Long polling, websockets y server-sent events: comunicación para el envío de datos en tiempo real," Mikarimin. Revista Científica Multidisciplinaria, vol. 10, no. 3, pp. 101–120, 2024, doi: 10.61154/mrcm.v10i3.3272.
- [4] E. C. Nugroho, "A Horizontally Scalable WebSocket Architecture for Cost-Effective Online Examination Proctoring System on AWS Cloud Infrastructure," Engineering, Mathematics and Computer Science Journal (EMACS), vol. 7, no. 1, pp. 109–120, 2025, doi: 10.21512/emacsjournal.v7i1.12770.
- [5] A. Morchid et al., "IoT-enabled smart agriculture for improving water management: A smart irrigation control using embedded systems and Server-Sent Events," Scientific African, vol. 27, Art. no. e02527, Mar. 2025, doi: 10.1016/j.sciaf.2024.e02527.
- [6] R. Duan and Z. K. Ma, "Qianji: A resilient framework for orchestrating 'A Thousand Machines' in distributed performance," in Proc. Int. Conf. New Interfaces for Musical Expression (NIME 2026), London, U.K., Jun. 2026, pp. 1–7. [Online]. Available: https://nime.org/proceedings/2026/nime2026_25.pdf. Accessed: Oct. 6, 2026.
- [7] R. Katagiri, T. Sone, S. Aihara, and T. Kawamura, "A resilient and extensible Digital Curling platform using Server-Sent Events and REST APIs," Applied Sciences, vol. 16, no. 14, Art. no. 6886, Jul. 2026, doi: 10.3390/app16146886.

- [8] W. Roome and Y. Yang, "Application-Layer Traffic Optimization (ALTO) incremental updates using Server-Sent Events (SSE)," Internet Engineering Task Force, RFC 8895, Nov. 2020, doi: 10.17487/RFC8895.
- [9] A. Savidis, *Real-Time Train Tracking over Satellite Backhaul: A GNSS/INS System with HMM Map Matching and Graph-Aware Dead Reckoning for the Greek Railway Network*. Heraklion, Greece: University of Crete, Tech. Rep., 2026. [Online]. Available: <https://zenodo.org/records/20257402>. Accessed: Oct. 6, 2026.
- [10] Redis, "Redis Pub/Sub," *Redis Documentation*. [Online]. Available: <https://redis.io/docs/latest/develop/pubsub/>. Accessed: Oct. 6, 2026.